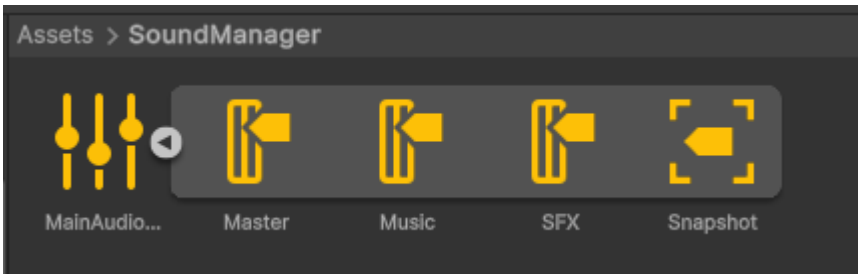
Musique & Sons

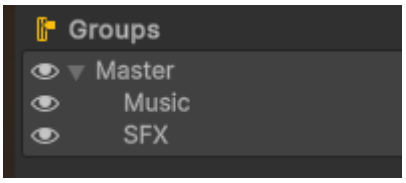
Musique et Sons

The project includes a sound mixer:
Location: Assets/SoundManager



The sound mixer is used to separate different sounds (sound effects, music, etc.).

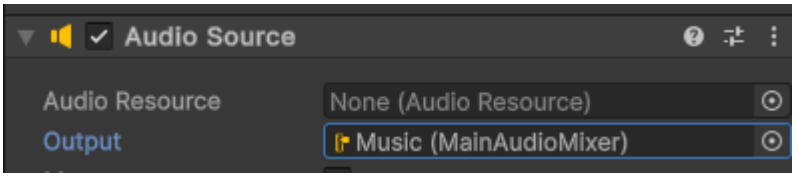
There are three elements for managing music:



Les SFX, sont les effets sonores.

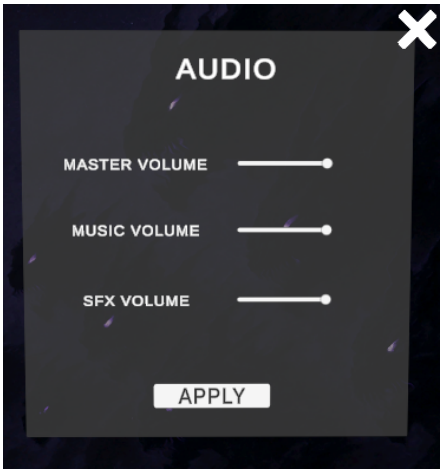
Creating a Component with Music

When creating a component with music (Component -> Audio -> Audio Source):



The Output must be set to one of the following groups : Music or SFX.

Sounds can be managed directly in the game settings:



Class Units & Prefab Entités

Class Units et ses dérivés

Units class is the parent of all entity

```
public class Units : MonoBehaviour
```

All units must be derived, for example:

```
public class Keronas : Units
```

All units will look like this:

```
public class Cavalier : Units
{
   protected override void Start()
   {
       speed = 1f;
       attackRate = 1f;
       totalHealth = 6 * multiplierTotalHp;
       damagePerAttack = 4;
       defense = 3;

       manaCost = 4;
       team = UnitsTeam.Player;
       entityType = EntityType.Basic;
       base.Start();
   }
}
```

The rest of the work (such as moving entities) will be done in the Units class.

Depending on the entity, it will have either Capacities or Passives.
All you have to do is override the Capacite() and Passif() functions of the parent class.

```
public class Keronas : Units
{
   protected override void Capacite()
   {
      // do something
   }
   protected override void Passif()
   {
      // do something
   }
}
```

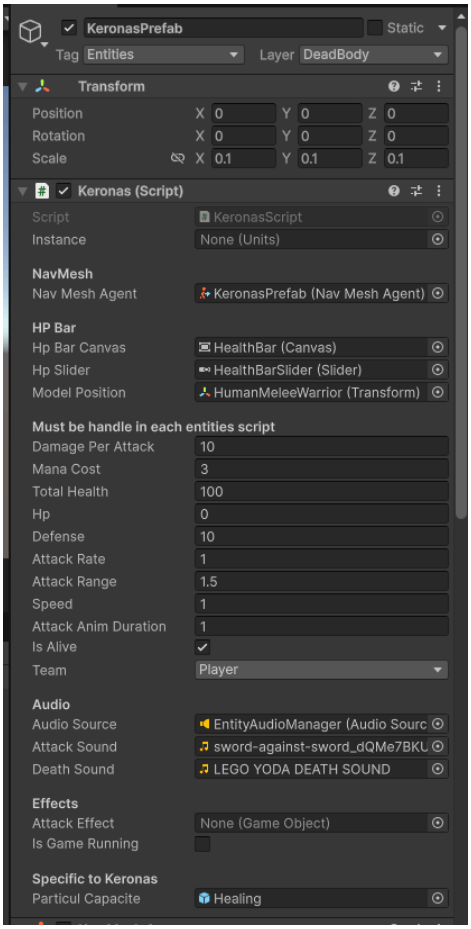
Prefab

To create an entity prefab, you need:
- A parent GameObject (KeronasPrefab, ChevalierPrefab, etc.),
- A Canvas for the HP bar (use the HealthBar prefab),
- The Champion’s 3D model (Entity Model).

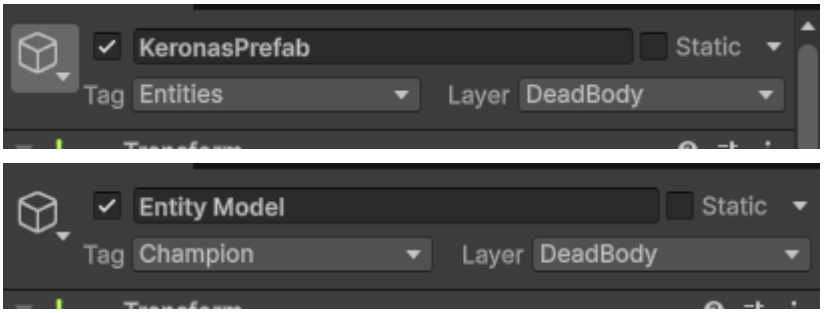
Other elements can also be added as needed.
Entity prefabs (Champions & Enemies) can be copied and pasted from the EntityPrefab prefab located in Assets/Prefab.

Here is the prefab for the Keronas entity:

For example: KeronasPrefab has the following entity script:

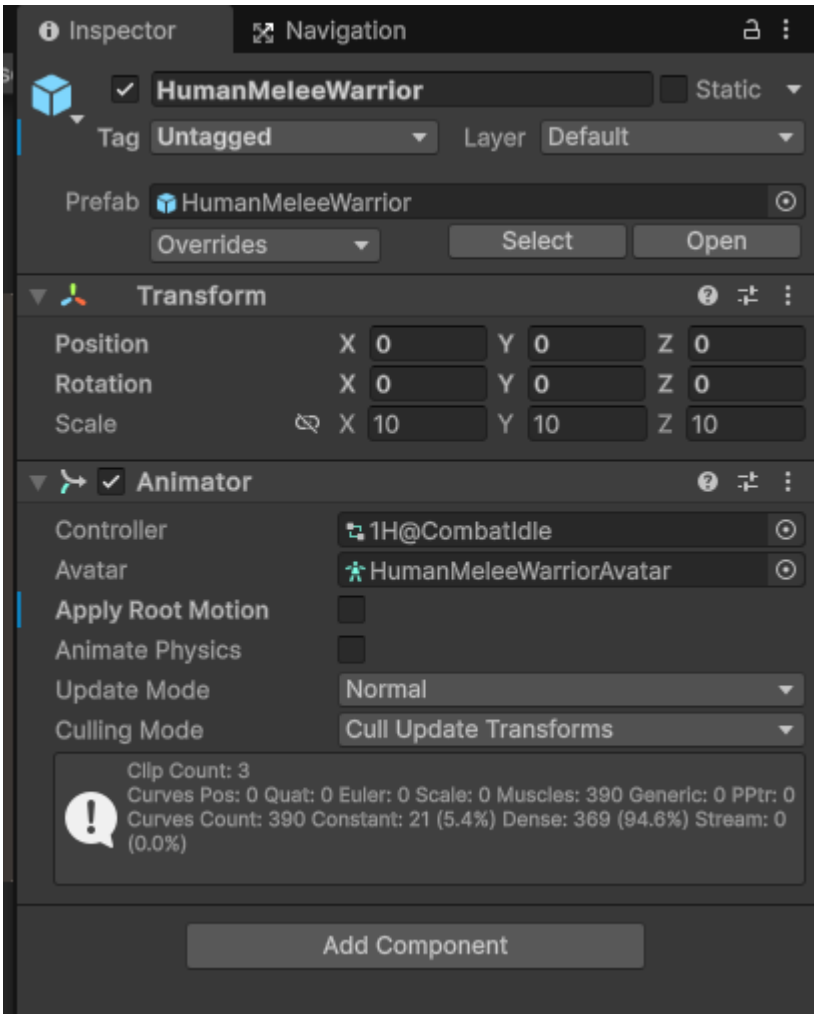


In the prefabs, you need to add a **Tag** to the **Entity Model** GameObject : “Champion”.
For the parent GameObject, set it to “Entities”:



Note: You don't need to adjust the **Layer**, it will be updated automatically during the game.

In the 3D model's Animator, make sure “Apply Root Motion” is disabled. Otherwise, the entity will move twice as fast as intended:



Bug Tracker

Class BugTracker

BugTracker will log everything that happens in the game to a log file.

How do you use it ?
Here are the four key features:

```
public static void Error(string msg); // qlq chose qui se bloque, pas forcément de crash.
public static void Warning(string msg); // qlq chose d'anormal mais qui ne bloque pas le jeu.
public static void Info(string msg);
public static void Critical(string msg); // erreur fatale, l'app peut planter
```

They allow you to write different types of information with different severity levels to the log file.

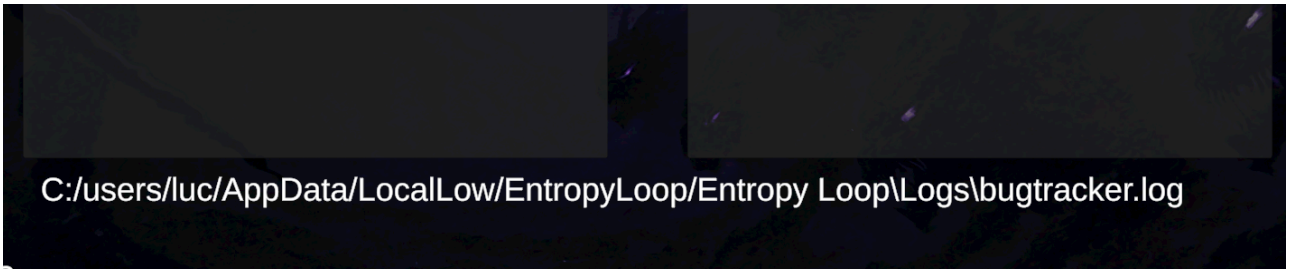
Here is an example of implementation:

```
protected virtual void Start() {
       BugTracker.Info("[Units] New entity '" + gameObject.name + "' created.");
   }
```

Here the output :

```
[2025-11-16 17:09:10] [Info]: New entity 'KeronasPrefab' created.
```

Be sure to include as much information as possible.
You can find the path to the bugtracker.log file from the console in Unity or in the settings when you launch the game.



Animated Prefab Creation Menu

Animated Prefab Creation Menu

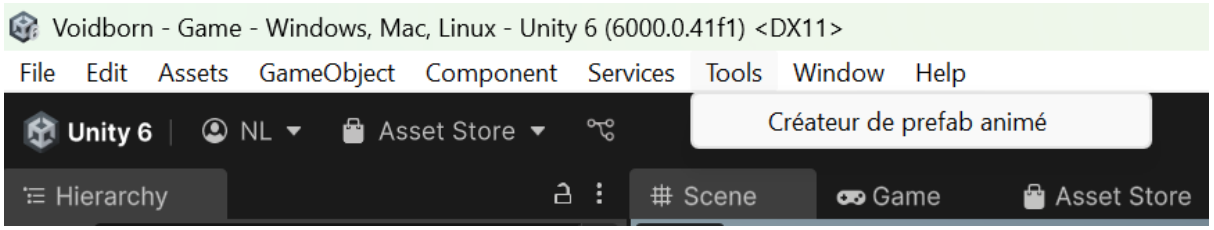
This tool is designed to automate the creation of a playable Prefab (with a configured Animator Controller) from:

- a 3D model (.fbx or Prefab),
- animations (Idle/Run required, Attack/Die optional).

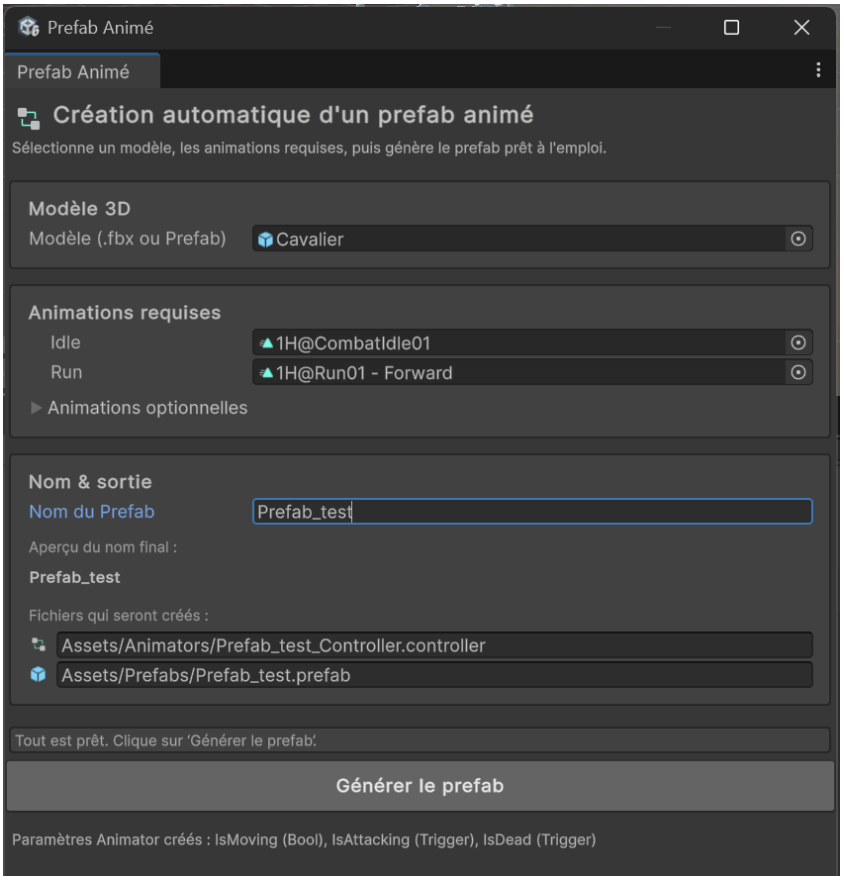
The tool generates:

- an Animator Controller with parameters and transitions,
- a Prefab ready to drop into the scene.

Click **Tools/Animated Prefab Creator**.



Select a 3D model and animations, then click “Generate Prefab.”
→ A ready-to-use animated prefab will be created.



Loading screen

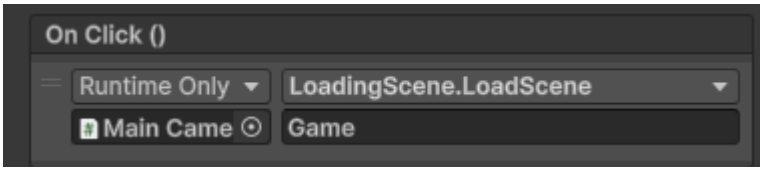
Loading screen

Here the **LoadingScene** class :

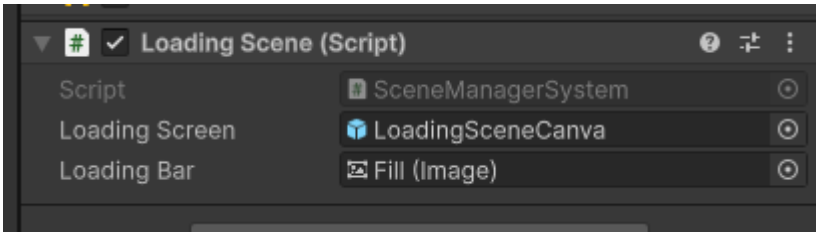
```
public class LoadingScene : MonoBehaviour
{
    public GameObject loadingScreen;
    public Image loadingBar;

    public void ChangeScene(string scene) // Change scene WITHOUT a loading
screen
    public void LoadScene(string scene) // Change scene WITH a loading screen
    IEnumerator LoadSceneAsync(string scene)
    public void QuitGame()
}
```

All you have to do is use it in a button and enter the name of the next scene.
You can use ChangeScene or LoadScene, all depending on the loading time.

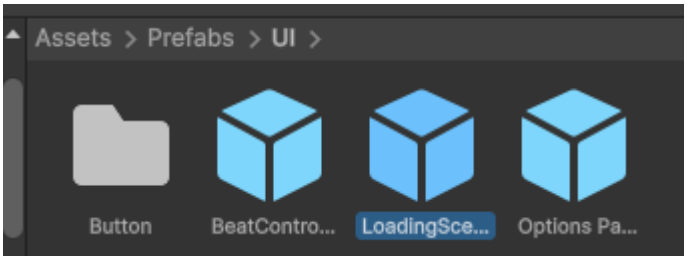
WARNING: LoadScene may slow down performance



You need to add this component to a GameObject. It is often attached to the camera.



The **loading screen** and **loading bar** can be retrieved from the **LoadingSceneCanva** prefab.



Deck management

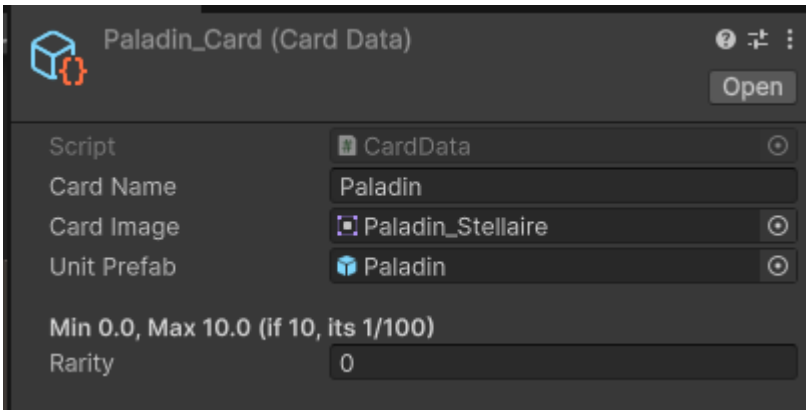
Deck management

Comment créer une carte ?
Il faut faire **Clic droit -> Create -> Card**

Voici la class **CardData** :

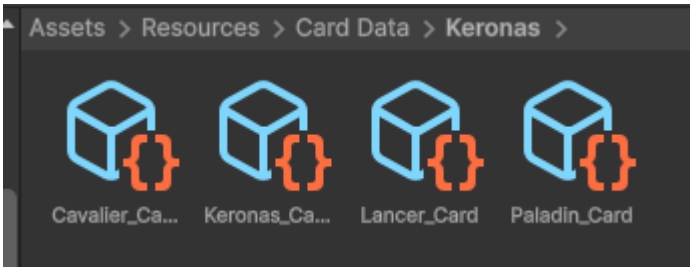
```
[CreateAssetMenu(fileName = "NewCard", menuName = "Card")]
public class CardData : ScriptableObject
{
    public string cardName;
    public Sprite cardImage;
    public GameObject unitPrefab;
    [Header("Min 0.0, Max 10.0 (if 10, its 1/100)")]
    public float rarity;
}
```

Tu obtiens une **Card Data !**



Pour **Card Image**, il faut donner le sprite de la carte qui se trouve ici : **Assets/Art/Decks/nom du deck**

Dans l'image ci-dessous, on est dans le dossier Ressources. C'est ici que la Class **DeckManager** va chercher et load les cards.



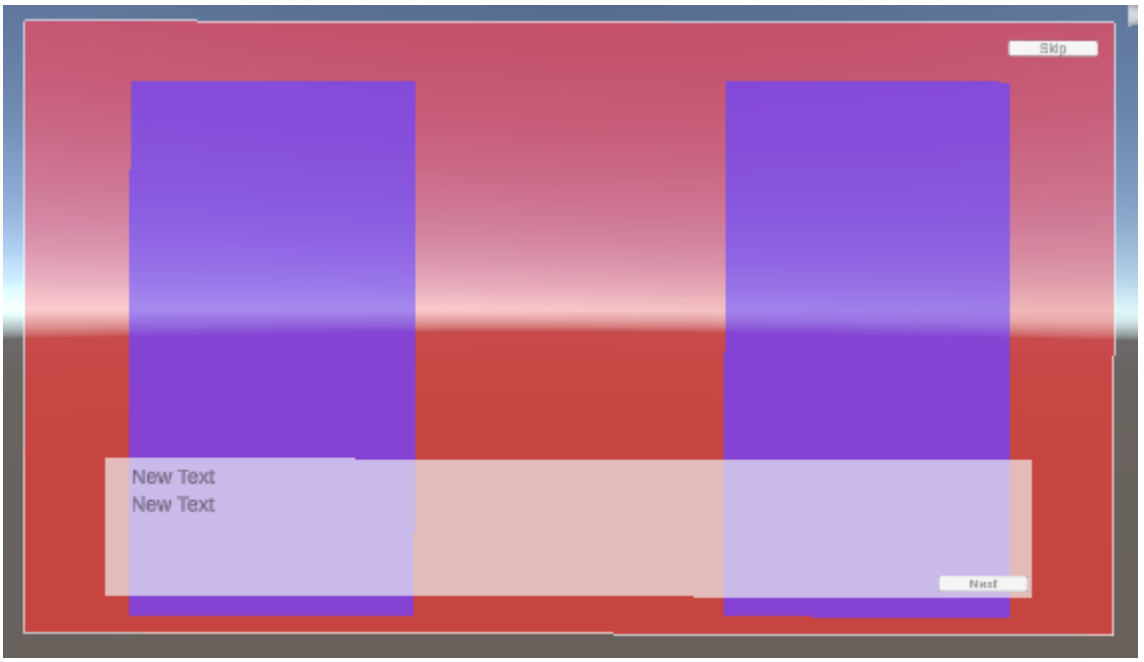
Il suffit juste de créer un dossier et de mettre les cards liés au deck dedans.

Story Manager

Story Manager

There is a specific section for Stories.

The canvas looks like this:



red: background
blue: characters on the left and right
white rectangle: speech bubble
skip & next buttons
(In the scene, the colors are white. But to show their locations, they have been colored for documentation purposes)

File structure:

Story/
- **Chapter1/**
  - **chpt1-1.json**
  - **chpt1-2.json**
  - **…**
- **Chapter2/**
  - **…**
- **ChapterX/**
  - **…**

Chapter files will be automatically loaded based on the level.

```
{
   "dialogues": [
   {
       "backgroundName": "Background/forest",
       "charLeftImg": "",
       "charRightImg": "",
       "speakerName": "Narrateur",
       "text": "Keronas entre dans la jungle. Silence pesant. La végétation
est décolorée, tordue, immobile."
   },
   {
       "backgroundName": "Background/forest",
       "charLeftImg": "Keronas/Keronas_idle",
       "charRightImg": "",
       "speakerName": "Keronas",
       "text": "Ce n'est plus Verdentia…"
   },
}
```

Here is an example of the dialogues found in a file named “chpt1-1.json.”
It includes the background image, the characters on the left and right, the speaker, and the text.

The StoryManager class retrieves information directly from the GameManager class instance.

Game Manager

Game Manager

The GameManager class is a “DontDestroyOnLoad” & singleton class.

It is used to pause or resume the game with the function PauseGame() & ResumeGame().

The class stores all information related to the levels. It retrieves this information directly from the “Levels/levels_config.json” file.

```
{
   "levels": [
   {
       "currentlevel": 1,
       "chaptersBeforeGame": "",
       "chaptersAfterGame": "",
       "spawnAlgo": {
           "spawnStartegy": [0],
           "deck": [""],
           "manaCost": [9],
           "difficulty": "easy"
       }
   },
   {
       "currentlevel": 2,
       "chaptersBeforeGame": "Chapter1/chpt1-1",
       "chaptersAfterGame": "Chapter1/chpt1-1aft",
       "spawnAlgo": {
           "spawnStartegy": [0],
           "deck": [""],
           "manaCost": [3],
           "difficulty": "easy"
       }
   },
}
```

The data is then retrieved into `LevelData[]`.
`LevelData[]` is then automatically managed in GameManager, and the data can be accessed from the class instance.

Save System

Save System

Here are the 3 main functions of the class SaveSystem :

```
public static void Save(bool reset = false);
public static void Load();
public static void ResetFile();
```

These functions handle the player's data using the SaveData class :

```
public class SaveData
{
    public int currentLevel;
    public int maxLevelReach;
    public int maxLevelFinish;
    public DeckData selectedDeck;
}
```

The save file is stored under the name : GameData.json

The reset parameter in Save() function allows creating a save file after resetting all fields in SaveData back to default values.

ResetFile() deletes the existing GameData.json file from the disk and immediately calls Save(True) to re-initialize a clean save file.

GameLog Manager

Game Log System

1. Overview

The Game Log System allows any gameplay script to push a short message (with an optional icon) into a running log. The log is stored centrally, broadcast via an event, and rendered into a scrollable UI list. Players can open and close a log window to review recent activity.

1.1 Purpose

To give players an easy way to check the history of events without needing a custom UI display for each event. Any system in the game can call a single method to log a message.

1.2 Key Responsibilities

- Store a rolling history of game events, capped at a configurable maximum.
- Notify listeners the moment a new log entry is added.
- Render each new entry into a UI list, newest entry on top.
- Provide a simple open/close toggle for the log panel.

2. Architecture

[Any Gameplay Script]
|
| GameLogManager.Instance.AddLog(msg, icon)
v
[GameLogManager] --stores--> List<GameLogBase>
|
| Add_to_log event (Action<GameLogBase>)
v
[GameLogUI] --instantiates--> [LogTemplateUI] (one per entry)

[ActionLogWindow] --independently controls--> visibility of the log panel GameObject

Script	Role
GameLogBase.cs	Class used for the content of a message in the log.
GameLogManager.cs	Central singleton that stores log history and raises an event when a new entry is added.
GameLogUI.cs	Listens for new entries and spawns a UI row for each one.
LogTemplateUI.cs	Controls a single log row's UI (text + icon).
WindowLogAction.cs	Opens/closes the log panel.

3. Component Reference

3.1 GameLogBase

Represents a single log entry. Instances are created by GameLogManager and are otherwise not modifiable after creation.

Member	Type	Description
msg	string	The log message text shown to the player.
icon	Sprite	Optional icon shown alongside the message. May be null.
timelog	float	`Time.time` at the moment the entry was created. Set automatically by the constructor.
GameLogBase(msg, icon = null)	constructor	Creates the entry and stamps timelog with the current Time.time.

3.2 GameLogManager

Owns the authoritative list of log entries and is the single entry point other scripts call to record an event.

Fields

Member	Type	Description
Instance	static GameLogManager	Global access point, assigned in Awake().
logs	List<GameLogBase>	Full history, newest entry at index 0.
Add_to_log	event Action<GameLogBase>	Fired once per new entry, after it has been added to logs.
maxLogs	int (Inspector, default 50)	Maximum number of entries retained; oldest entries are discarded beyond this.

Methods

Method	Description
AddLog(string msg, Sprite icon = null)	Creates a new GameLogBase, inserts it at the front of logs, trims the oldest entry if the list exceeds maxLogs, then invokes Add_to_log.

3.3 GameLogUI

Bridges the manager's data event to the UI. Placed on the log panel's content view (typically inside a Scroll View).

Member	Type	Description
log_parent	Transform (Inspector)	Parent transform new log rows are instantiated under (typically a Scroll View's Content object).
log_template	GameObject (Inspector)	Prefab for a single log row; must have a LogTemplateUI component.

On Start(), it subscribes to `GameLogManager.Instance.Add_to_log`. Each time the event fires, it instantiates log_template under log_parent, calls Setup() on its LogTemplateUI component, and moves the new row to the top of the sibling order (`SetAsFirstSibling`) so the most recent entry is visually first.

3.4 LogTemplateUI

Controls the visual state of a single log row prefab instance.

Member	Type	Description
text	TMP_Text (Inspector)	Displays the log message.
icon	Image (Inspector)	Displays the log icon, if present.
Setup(GameLogBase log_added)	method	Populates text.text with the message; shows and assigns icon.sprite if an icon was provided, otherwise hides the icon's GameObject.

3.5 ActionLogWindow

A minimal open/close controller for the log panel, independent of the logging pipeline itself. Typically wired to a button's OnClick event.

Member	Type	Description
actionLogPanel	GameObject (Inspector)	The panel to show/hide (e.g. the window containing the GameLogUI content).
OpenWindow()	method	Activates actionLogPanel.
CloseWindow()	method	Deactivates actionLogPanel.

4. Data Flow — Adding a Log Entry

1. A gameplay script calls the line : `GameLogManager.Instance.AddLog("message", icon)`.
2. GameLogManager creates a new GameLogBase, stamping it with the current Time.time.
3. The entry is inserted at index 0 of logs (newest first).
4. If logs.Count exceeds maxLogs, the oldest entry (last index) is removed.
5. GameLogManager raises Add_to_log, passing the new entry.
6. GameLogUI, subscribed to that event, instantiates a log_template instance under log_parent.
7. The new instance's LogTemplateUI.Setup() is called, populating its text and icon.
8. The instance is moved to the top of the UI list via SetAsFirstSibling().

5. Usage Example

```
GameLogManager.Instance.AddLog("Picked up Iron Sword");
```

Mana System

Mana System

1. Overview

Mana is a capped numeric resource (0–maxMana) held by the player. Placing a unit from a card onto the grid costs mana equal to that card's `manaCost`. The system is intentionally simple: a central manager holds the value and clamps it, a UI script renders it as discrete segments, and consumer scripts (cards, grid cells) check and spend it as part of the unit-placement flow.

1.1 Purpose

To provide a single authoritative mana value the player spends against when deploying units, with a visual bar that always reflects the current amount, and a small API (`HasEnoughMana`, `SpendMana`, `AddMana`) that other systems call into rather than manipulating mana directly.

1.2 Key Responsibilities

- Hold the current/max mana value and keep it clamped to a valid range.
- Expose a simple check-then-spend API for other systems (primarily unit placement).
- Reflect the current mana visually as a segmented bar.
- Allow mana to be spent when a card is played, using the cost defined on that card.

2. Architecture

```
[ManaManager]  --owns--> currentMana / maxMana
      |
      |  UpdateUI() on every change
      v
[ManaBarUI]  --renders--> segment colors (active/inactive)

[CardData] (per-card asset)        [UnitData] (per-unit asset)
      |  manaCost                        |  manaCost (duplicate stat)
      v                                  v
[GridCell.OnMouseDown()]  --checks ManaManager.HasEnoughMana()-->
                           --spawns unit, then calls ManaManager.SpendMana()-->
```

Script	Role	Depth in this doc
ManaManager.cs	Central singleton holding and clamping the mana value; exposes spend/add/check API.	Full
ManaBarUI.cs	Renders currentMana as a row of colored segments.	Full
CardData.cs	ScriptableObject defining a playable card's stats, including its manaCost.	Summary
UnitData.cs	ScriptableObject defining a unit's combat stats, including its own manaCost.	Summary
GridCell.cs	Handles clicking a board cell: validates mana, spawns the unit, spends mana.	Summary

3. Core Components

3.1 ManaManager

Holds the player's current mana and is the only script that should mutate it directly.

Member	Type	Description
currentMana	int (default 3)	Current mana value.
maxMana	int (default 10)	Upper clamp for currentMana.
manaBarUI	ManaBarUI (Inspector)	UI reference updated whenever mana changes.
instance	static ManaManager	Global access point, assigned in Start().

Methods

Method	Description
HasEnoughMana(int amount)	Returns whether currentMana >= amount. Read-only check, does not spend.
AddMana(int nbrToAdd = 1)	Adds mana, clamped to [0, maxMana], then refreshes UI.
RemoveMana()	Removes exactly 1 mana, clamped to [0, maxMana], then refreshes UI.
SpendMana(int amount)	Subtracts an arbitrary amount, clamps to [0, maxMana], then refreshes UI.
UpdateUI() (private)	Pushes currentMana to manaBarUI if assigned.

3.2 ManaBarUI

Purely visual — a fixed array of Image segments, one per mana point, colored active or inactive based on the current value.

Member	Type	Description
segments	Image[] (Inspector)	One image per mana point/segment; array length should match maxMana.
activeColor	Color (default cyan)	Color for segments within the current mana amount.
inactiveColor	Color (default gray)	Color for segments beyond the current mana amount.
UpdateMana(int currentMana)	method	Colors segments[0..currentMana) active and the rest inactive.

4. Consumer Scripts

CardData.cs — ScriptableObject (`[CreateAssetMenu]`) representing a playable card in hand: name, image, color, unit prefab, `manaCost`, and a separate `goldCost`. Pure data, no logic.

UnitData.cs — ScriptableObject representing a unit's full stat block (health, damage, attack rate/range, speed, entity type, class list, reward, difficulty), which also carries its own `manaCost`. Appears intended as the data-driven replacement for the hardcoded stats currently set in `Units.LoadInformationsFromUnitData()` (see doc for Units.cs), though that method isn't wired to read from a UnitData asset yet.

GridCell.cs — Handles the click-to-place flow on a board tile:

1. On mouse down, bail out if the click is blocked (game currently running, or pointer over UI tagged `BlockClick`).
2. If a card is selected (`GameLoopManager.instance.selectedCard`), read its `manaCost` and check `ManaManager.HasEnoughMana(cost)`.
3. If insufficient, do nothing. If sufficient, spawn the unit prefab at the cell, register it with `GameLoopManager`, then call `ManaManager.SpendMana(cost)` and clear/deselect the hand slot.
4. If no card is selected, show a "No card selected" floating text instead.

Also handles simple visual highlighting of the cell (`SetHighlight`) independent of the mana flow.

5. Data Flow — Spending Mana to Place a Unit

1. Player selects a card in hand (selection state lives in `GameLoopManager`, not shown here).
2. Player clicks a `GridCell`.
3. `GridCell.OnMouseDown()` checks the click isn't blocked by UI or an active game state.
4. It reads `cardData.manaCost` from the selected card.
5. It calls `ManaManager.HasEnoughMana(cost)`. If false, the click is a no-op.
6. If true, it spawns the unit prefab at the cell and registers it.
7. It calls `ManaManager.SpendMana(cost)`, which clamps currentMana and calls `UpdateUI()`.
8. `ManaBarUI.UpdateMana()` recolors the segments to match the new value.
9. The hand slot is cleared and the card deselected.

6. Setup / Integration Checklist

- Place a single ManaManager in the scene, with `manaBarUI` assigned to the UI object holding the segment Images.
- Size the `segments` array on ManaBarUI to match the intended `maxMana`, or keep them in sync manually — the script does not resize the array itself.
- Author card costs via CardData assets (`manaCost`) — this is what GridCell actually reads at placement time.
- If migrating units to data-driven stats, wire `UnitData.manaCost` (and other stats) into `Units` via `LoadInformationsFromUnitData()`, which is currently hardcoded rather than reading from an asset.

Abilities System

Unit Abilities System

1. Overview

Every unit can optionally have one active ability and/or one passive ability. Both are implemented as `virtual` methods on `Units` (`Capacite()` and `Passif()`) that do nothing by default but using subclasses to override them to add behavior.

1.1 Purpose

To let each unit define unique gameplay behavior without touching the shared `Units` update loop, by hooking into two well-defined extension points.

1.2 Key Responsibilities

- `Units` tracks a capacity gauge (`capacityPoints` / `capacityTriggerMax`) that fills from combat actions and triggers `Capacite()` when full — this is the **active ability**.
- `Units` calls `Passif()` every frame while the game is running, leaving cooldown/trigger logic entirely to the subclass — this is the **passive ability**.
- Subclasses override `Start()` to set their own base stats before calling `base.Start()`.

2. Architecture

```
[Units] (base class)
  |
  |-- capacityPoints, capacityTriggerMax, EntityHasCapacity   (active ability
state)
  |-- GainCapacityPoints(pts)   <-- called from TakeDamage() and Attack()
  |-- Update(): if EntityHasCapacity && capacityPoints >= capacityTriggerMax ->
Capacite()
  |-- Passif()   <-- called unconditionally every Update() while isGameRunning
  |
  |-- virtual Capacite() { }   (no-op by default)
  |-- virtual Passif()   { }   (no-op by default)
        ^                            ^
        |  override                  |  override
[Keronas : Units]            [Paladin : Units]
  - Active ability             - Passive ability
  - Buffs allied units          - Periodic damage-block window
    for a timed duration          via protectedHits
```

Script	Role
Units.cs	Defines the capacity gauge, the `Capacite()`/`Passif()` hooks, and when each fires. (Covered in a separate doc; summarized here for context.)
Keronas.cs	Example **active** ability: a team buff triggered once the capacity gauge fills.
Paladin.cs	Example **passive** ability: a recurring, self-timed damage-block window.

3. Base Hooks (from Units.cs)

3.1 Active ability — Capacite()

Member	Type	Description
EntityHasCapacity	protected bool (default false)	Must be set `true` by a subclass for the capacity gauge to ever trigger `Capacite()`.
capacityPoints	protected float	Current gauge fill, starts at 0, reset to 0 in `Start()` and again each time `Capacite()` fires.
capacityTriggerMax	protected float (default 100)	Threshold at which the gauge triggers the ability.
GainCapacityPoints(float pts)	private method	Adds `pts` to `capacityPoints`, only while it's below 100. Called from `TakeDamage()` (scaled by defense-based reduction) and `Attack()` (scaled by `damagePerAttack * 3`).
Capacite()	protected virtual method	No-op in the base class. Checked every `Update()`: if `EntityHasCapacity` and `capacityPoints >= capacityTriggerMax`, the gauge resets to 0 and `Capacite()` is invoked.

So the active ability is gauge-driven: it charges from both dealing and taking damage, and fires automatically once full, the subclass only needs to define the effect, not the trigger condition.

3.2 Passive ability — Passif()

Member	Type	Description
passifTimer	protected float	Free-form timer field; not touched by the base class outside declaration — subclasses manage it entirely themselves.
Passif()	protected virtual method	No-op in the base class. Called unconditionally on every `Update()` while `isGameRunning` is true.

Unlike the active ability, the base class provides no cooldown or condition logic for `Passif()`, every subclass that overrides it is responsible for its own timing (as `Paladin` does with `passifTimer`).

4. Keronas — Active Ability Example

Setup (Start(), before base.Start()): speed 1, attack rate 1.5, health `8 * multiplierTotalHp`, damage 4, defense 4, mana cost 5, team Player, entity type Champion, `EntityHasCapacity = true`.

Capacite() — "team buff":

1. Resets `capacityPoints` to 0 and logs the activation via `GameLogManager`.
2. Finds all GameObjects tagged `"Champion"` and, for each living one, adds +10 `damagePerAttack`, +1 `attackRate`, and +2 `totalHealth`, and spawns a particle effect at Keronas's position for each buffed unit.
3. Schedules `ResetDamagePerAttack()` via `Invoke(..., 30f)`, which walks the same tagged units 30 seconds later and subtracts the same amounts, undoing the buff.

`ResetDamagePerAttack()` is a plain private method that reverses step 2's stat changes on the same tag group.

5. Paladin — Passive Ability Example

Setup (Start(), before base.Start()): speed 1, attack rate 0.7, health `4 * multiplierTotalHp`, damage 2, defense 4, mana cost 3, team Player, entity type Basic. (`EntityHasCapacity` is left `false` — Paladin has no active ability.)

Member	Type	Description
protectionCooldown	float (Inspector, default 12)	Seconds between passive activations.
protectionHits	int (Inspector, default 4)	Number of hits granted per activation.

Passif() — "Protection":

1. Accumulates `passifTimer += Time.deltaTime` every frame.
2. Once `passifTimer >= protectionCooldown`, resets the timer to 0 and sets `protectedHits = protectionHits` (clamped to a max of 4 via `Mathf.Min`), then logs the activation.

The actual effect of `protectedHits` is consumed elsewhere, in the base class's `TakeDamage()`: each point of incoming positive damage while `protectedHits > 0` is fully blocked (0 damage dealt) and decrements the counter by one, until it reaches 0.

6. Data Flow

Active ability (Keronas-style):

1. Keronas deals or takes damage during normal combat (`Attack()` / `TakeDamage()` in `Units`).
2. `GainCapacityPoints()` adds to `capacityPoints`, capped at 100.
3. `Units.Update()` sees `capacityPoints >= capacityTriggerMax`, resets the gauge, and calls `Capacite()`.
4. Keronas's override buffs all `"Champion"`-tagged units and schedules the buff's expiry 30 seconds later.

Passive ability (Paladin-style):

1. Every frame, while the game is running, `Units.Update()` calls `Passif()` unconditionally.
2. Paladin's override advances its own `passifTimer` and only acts once it exceeds `protectionCooldown`.
3. On activation, it grants `protectedHits`, which are later spent one-for-one by `TakeDamage()` in the base class to negate incoming hits.

Génération de carte & Systèmes Caméra

Génération de carte & Systèmes Caméra

VoidMapGeneratorGPU · CameraOcclusionTransparency · CameraManager

Moteur : Unity (URP / Built-in)**Langage** : C#**Branche** : feat/maps**Date** : Juillet 2026

1. VoidMapGeneratorGPU

`Singleton[ExecuteAlways][RequireComponent(NavMeshSurface)]`

Composant principal de la génération procédurale. Génère une île flottante complète : terrain mesh, forêt, village, lac, château, routes, arène centrale, NavMesh et spawn des ennemis. Toujours présent dans la scène Game. Tous les autres systèmes en dépendent.

Démarrage automatique

Si `generateOnPlay` est activé dans l'Inspector, la carte se génère automatiquement au Play. Rien à appeler.

Changer la seed

Une seed différente produit une île entièrement différente. Relance automatiquement une génération complète.

```
// Génère une nouvelle île avec la seed donnée
VoidMapGeneratorGPU.instance.SetSeed(67890);
```

Changer la phase

La phase contrôle le niveau de développement et de corruption de l'île (0 = nature vierge → 5 = consumée par le Néant). Relance automatiquement la génération.

```
// Active le château et le lac
VoidMapGeneratorGPU.instance.SetPhase(3);
```

Phase	Contenu activé
0	Île + forêt + arène uniquement
1	+ Village + routes
2	+ Lac

3	+ Château
4 – 5	Corruption croissante du Néant (fissures, trou noir, atmosphère violette)

Lire le terrain depuis le code

```
// Hauteur du terrain à la position (x, z)
float h = VoidMapGeneratorGPU.instance.TerrainHeight(x, z);

// Tester si une position est dans le lac
bool dansLac = VoidMapGeneratorGPU.instance.IsInsideLake(transform.position);

// Tester si une position est dans une zone corrompue
bool corrompu = VoidMapGeneratorGPU.instance.IsInVoidBite(transform.position);

// Niveau de corruption entre 0 (sain) et 1 (totalement corrompu)
float corruption = VoidMapGeneratorGPU.instance.CorruptionAt(transform.position);
```

Paramètres Inspector

Champ	Défaut	Description
seed	12345	Graine de génération. Chaque valeur produit une île unique
phase	0	Niveau de développement / corruption (0–5)
generateOnPlay	true	Génère automatiquement la carte au démarrage Play
islandRadius	38	Rayon de l'île en unités Unity
chessboardSize	3	Taille de la grille d'arène
rebuildNavMeshAfterGenerate	true	Reconstruit le NavMesh après chaque génération
autoClearBeforeGenerate	true	Supprime la génération précédente avant d'en créer une nouvelle

Pipeline de génération (GenerateAsync)

BuildMaterials	Crée ~30 matériaux procéduraux + texture étoilée du Néant
ComputeVoidBites	Calcule les cratères du Néant (déterministe selon la seed)
BuildIslandHybrid	Mesh terrain CPU (collider + NavMesh) + ventre rocheux + rochers
BuildParkAndArena	Socle de l'arène, dalles avec composant GridCell, décoration
BuildVillage	Phase ≥ 1 — maisons procédurales par Poisson Disk Sampling
BuildForest	Arbres procéduraux par Poisson Disk. Ajoute TreeOccluder sur chaque arbre
BuildLake	Phase ≥ 2 — plan d'eau avec matériau transparent
BuildCastle	Phase ≥ 3 — enceinte circulaire avec tours et merlons
BuildRoads	Phase ≥ 1 — branches radiales + anneau périphérique
VoidMapPropsGPU.GenerateProps	Props décoratifs (buissons, meubles, lucioles, nuages...)
NavMesh.BuildNavMesh	Reconstruction du NavMesh sur toute la géométrie
EnemySpawnAlgo.SpawnEnemies	Spawn des ennemis selon la stratégie du niveau (Play uniquement)
AutoPostProcess.Apply	Post-processing visuel automatique (fog, lumière ambiante)

---

2. TreeOccluder

Composant marqueur vide. Ajouté automatiquement sur chaque arbre lors de l'étape **BuildForest** du générateur. Ne jamais l'ajouter manuellement.

Son unique rôle est de permettre à `CameraOcclusionTransparency` de retrouver tous les arbres de la scène sans tag Unity, via :

```
FindObjectsByType<TreeOccluder>(FindObjectsSortMode.None);
```

La liste est rafraîchie automatiquement après chaque génération de carte.

---

3. CameraOcclusionTransparency

`Singleton`

Attaché à la gameplay camera. Rend transparents les arbres qui passent entre la caméra et l'arène, pour que la grille reste toujours visible. Fonctionne sans collider sur les arbres — détection géométrique pure.

Fonctionnement automatique — Ce script tourne tout seul en LateUpdate. Il suffit que `arenaCenter` soit assigné pour qu'il soit actif. Aucun appel manuel nécessaire en temps normal.

Assignation dans l'Inspector

Champ	Défaut	Description

`arenaCenter`	—	**Obligatoire.** Transform positionné au centre de la grille d'arène
`arenaRadius`	5	Rayon de détection autour de l'arène. Augmenter si la grille est grande
`fadeAlpha`	0.2	Opacité des arbres occultants (0 = invisible, 1 = opaque)
`fadeSpeed`	8	Vitesse du fondu en alpha/seconde
`detectionRadius`	2	Épaisseur de détection autour de chaque arbre

Contrôle depuis le code

```
// Changer la cible de l'arène depuis le code (utile après une re-génération)
CameraOcclusionTransparency.instance.SetArenaCenter(monTransform);

// Ajuster le rayon si la taille de la grille change
CameraOcclusionTransparency.instance.SetArenaRadius(7f);

// Rendre les arbres plus ou moins transparents
CameraOcclusionTransparency.instance.SetFadeAlpha(0.05f);
```

Algorithme de détection

La caméra vérifie 5 points de l'arène à chaque frame — le centre et les 4 coins. Si un arbre se trouve géométriquement entre la caméra et l'un de ces points, il reçoit un fondu progressif. Dès qu'il ne bloque plus aucun point, il redevient opaque.

```
// Points vérifiés (calculés automatiquement depuis arenaCenter + arenaRadius)
centre de l'arène
coin avant-droit    (+ arenaRadius, 0, + arenaRadius)
coin avant-gauche   (- arenaRadius, 0, + arenaRadius)
coin arrière-droit  (+ arenaRadius, 0, - arenaRadius)
coin arrière-gauche (- arenaRadius, 0, - arenaRadius)
```

Intégration avec la génération de carte

Pour éviter d'assigner `arenaCenter` à la main dans l'Inspector après chaque nouvelle scène, il est possible de l'appeler à la fin de `GenerateAsync` dans `VoidMapGeneratorGPU` :

```
// À la fin de GenerateAsync dans VoidMapGeneratorGPU.cs
if (CameraOcclusionTransparency.instance != null)
    CameraOcclusionTransparency.instance.SetArenaCenter(arenaCenter);
```

---

4. CameraManager

`Singleton`

Gère le basculement entre la caméra de jeu et la caméra libre de debug. Centralise également la gestion du curseur (lock / visible).

Comportement par défaut

La touche **C** bascule entre les deux caméras. La caméra de jeu est active par défaut au démarrage.

API publique

```
// Forcer la caméra de jeu (curseur visible, déverrouillé)
CameraManager.instance.SetGameplayCamera();

// Forcer la caméra libre debug (curseur verrouillé, FPS WASD + souris)
CameraManager.instance.SetFreeCamera();

// Basculer entre les deux
CameraManager.instance.ToggleCamera();

// Vérifier l'état actuel
bool enModeLibre = CameraManager.instance.IsFreeCameraActive();
```

Exemple d'utilisation — Bloquer la caméra libre pendant le combat

```
void StartOrStopCombat(bool combatActif)
{
    if (combatActif)
        CameraManager.instance.SetGameplayCamera(); // empêche le debug pendant le combat

    isGameRunning = combatActif;
}
```

---

5. Ordre d'initialisation à respecter

VoidMapGeneratorGPU	S'initialise en Awake (singleton). Génère en Start si generateOnPlay = true
VoidMapPropsGPU	Appelé automatiquement en fin de GenerateAsync. Aucune action requise
TreeOccluder	Ajouté automatiquement sur chaque arbre pendant BuildForest
NavMesh	Reconstruit automatiquement après la géométrie si rebuildNavMeshAfterGenerate = true
EnemySpawnAlgo	Appelé automatiquement en fin de génération (Play uniquement)
CameraOcclusionTransparency	Actif dès que arenaCenter est assigné. Se rafraîchit automatiquement après chaque génération
CameraManager	Indépendant — actif dès le démarrage de la scène